

LAPORAN ANALISIS PROGRAM

TUGAS REMEDIAL MATA KULIAH PEMROGRAMAN BERBASIS OBJEK (JAVA)

SISTEM KLINIK SEDERHANA

(Java Desktop - Swing & MySQL)

Nama Mahasiswa	: M.Faturahman.N.M
NIM	: 20240040262
Kelas	: TI24M
Program Studi	: TEKNIK INFORMATIKA
Dosen Pengampu	: YULHAN WAHYUDIN, M.Kom

UNIVERSITAS NUSA PUTRA

2026

DAFTAR ISI

A. Deskripsi Program	3
1. Nama Aplikasi	3
2. Tujuan Aplikasi.....	3
3. Permasalahan yang Diselesaikan.....	3
B. Analisis Konsep PBO.....	4
1. Class dan Object	4
2. Inheritance	4
3. Polymorphism.....	5
4. Encapsulation	5
5. Abstract Class dan Interface	6
6. Package.....	6
7. Collection.....	7
8. String Method.....	7
9. Exception Handling.....	8
10. File Handling	8
11. Database.....	9
C. Refleksi Pengembangan	10
1. Mengapa Memilih Studi Kasus Tersebut.....	10
2. Bagian Program Paling Sulit Dibuat	10
3. Kesalahan (Error) yang Paling Sering Ditemui	10
4. Tiga Fitur Pengembangan Lanjutan	10
5. Konsep PBO yang Masih Perlu Dipelajari	11
D. Lampiran	12
1. Class Diagram	12
2. Struktur Database.....	13
3. Screenshot Aplikasi dan Penjelasan Menu.....	14
Penutup	16

A. Deskripsi Program

1. Nama Aplikasi

Aplikasi yang dibuat pada tugas remedial ini diberi nama “Sistem Klinik Sederhana”, yaitu aplikasi desktop berbasis Java Swing yang terhubung dengan basis data MySQL untuk mengelola operasional administratif klinik berskala kecil, mulai dari data pasien, data dokter, data obat, hingga transaksi kunjungan/pemeriksaan pasien.

2. Tujuan Aplikasi

Aplikasi ini dibangun dengan tujuan sebagai berikut:

- Menyediakan sistem pencatatan data pasien, dokter, dan obat secara terkomputerisasi sehingga menggantikan pencatatan manual berbasis kertas.
- Mempermudah proses pencatatan transaksi kunjungan pasien beserta resep obat yang diberikan, termasuk perhitungan biaya secara otomatis.
- Menerapkan konsep-konsep Pemrograman Berbasis Objek (Class, Object, Encapsulation, Inheritance, Polymorphism, Abstract Class, Interface, dan Package) secara nyata dalam sebuah studi kasus yang utuh.
- Menerapkan konsep Java lanjutan seperti Collection, String Method, Exception Handling, dan File Handling ke dalam satu aplikasi yang saling terintegrasi.
- Menyediakan mekanisme login bertingkat (admin dan petugas) untuk menjaga keamanan akses data.

3. Permasalahan yang Diselesaikan

Klinik berskala kecil pada umumnya masih mengelola data pasien, rekam kunjungan, dan stok obat secara manual menggunakan buku atau berkas terpisah. Cara ini menimbulkan beberapa permasalahan, di antaranya:

- Pencarian data pasien atau riwayat kunjungan yang lambat karena data tidak terpusat.
- Risiko kehilangan atau duplikasi data pasien akibat pencatatan manual.
- Tidak adanya kontrol stok obat yang akurat, sehingga stok obat dapat habis tanpa disadari atau tercatat keluar melebihi stok yang tersedia.
- Perhitungan biaya kunjungan (biaya periksa + biaya obat) yang rawan salah hitung jika dilakukan manual.
- Tidak adanya jejak aktivitas (log) yang bisa ditelusuri ketika terjadi kesalahan input data.

Aplikasi Sistem Klinik Sederhana menjawab permasalahan tersebut melalui basis data terpusat (MySQL), validasi stok obat otomatis sebelum transaksi resep disimpan (memanfaatkan Exception Handling), pencatatan log aktivitas ke dalam berkas teks (File Handling), serta fitur pencarian data pada setiap modul master.

B. Analisis Konsep PBO

Bagian ini menjelaskan bagaimana setiap konsep Pemrograman Berbasis Objek yang menjadi ketentuan tugas diterapkan pada aplikasi Sistem Klinik Sederhana, lengkap dengan lokasi kode, alasan penggunaan, dan manfaatnya terhadap aplikasi.

1. Class dan Object

Lokasi implementasi: package klinik.model (Pasien.java, Dokter.java, Obat.java, User.java, Kunjungan.java, ResepObat.java) dan package klinik.view (mis. FormPasien.java)

```
// FormPasien.java - membuat object baru dari class Pasien
Pasien pasienBaru = new Pasien(
    noRm, nama, jenisKelamin,
    tanggalLahir, alamat, noTelp
);
pasienDAO.tambah(pasienBaru);
```

Alasan penggunaan:

Setiap entitas nyata dalam domain klinik (pasien, dokter, obat, kunjungan, resep obat, pengguna sistem) dimodelkan sebagai class tersendiri agar data dan perilakunya terbungkus rapi dalam satu unit. Object dibuat setiap kali form GUI perlu merepresentasikan satu baris data, misalnya saat pengguna menekan tombol “Simpan” pada form pasien.

Manfaat bagi aplikasi:

Struktur program menjadi modular dan mudah dipelihara karena setiap perubahan pada satu entitas (misalnya penambahan field baru pada Pasien) cukup dilakukan di satu class saja tanpa memengaruhi class lain.

2. Inheritance (Pewarisan)

Lokasi implementasi: package klinik.model - class Pasien dan Dokter mewarisi class Person

```
public abstract class Person {
    private String nama;
    private String noTelp;
    // getter/setter ...
    public abstract String getInfo();
}

public class Pasien extends Person {
    private String noRm;
    // constructor memanggil super(nama, noTelp)
}

public class Dokter extends Person {
    private String spesialisasi;
}
```

Alasan penggunaan:

Pasien dan Dokter sama-sama memiliki atribut dasar seorang “orang”, yaitu nama dan nomor telepon. Untuk menghindari duplikasi kode, kedua atribut beserta getter/setter-nya diletakkan pada superclass Person, kemudian Pasien dan Dokter mewarisinya melalui extends.

Manfaat bagi aplikasi:

Kode menjadi lebih ringkas (tidak ada duplikasi field nama/noTelp), dan apabila suatu saat atribut umum seorang “orang” perlu ditambah (misalnya email), penambahan cukup dilakukan sekali di class Person.

3. Polymorphism

Lokasi implementasi: method getInfo() pada Pasien.java & Dokter.java, serta interface ICrud<T> yang diimplementasikan oleh lima class DAO berbeda

```
// Overriding - Polymorphism
// Pasien.java
@Override
public String getInfo() {
    return "[Pasien] " + noRm + " - " + getNama();
}

// Dokter.java
@Override
public String getInfo() {
    return "[Dokter] " + getNama() + " - Spesialis " + spesialisasi;
}
```

Alasan penggunaan:

Polymorphism diterapkan dalam dua bentuk. Pertama, overriding: method abstrak getInfo() pada Person di-override berbeda oleh Pasien dan Dokter sesuai kebutuhan tampilannya masing-masing. Kedua, melalui interface ICrud<T> yang diimplementasikan oleh PasienDAO, DokterDAO, ObatDAO, KunjunganDAO, dan UserDAO, masing-masing dengan query SQL dan logika berbeda namun dipanggil dengan kontrak method yang sama.

Manfaat bagi aplikasi:

Kode pemanggil (GUI) dapat memperlakukan objek Person secara umum tanpa perlu tahu apakah itu Pasien atau Dokter, dan setiap DAO dapat dikembangkan atau diganti implementasinya tanpa mengubah kode pada form yang memanggilnya (loose coupling).

4. Encapsulation

Lokasi implementasi: seluruh class pada package klinik.model, contoh: Person.java, Obat.java

```
public class Obat {
    private int idObat;
    private String namaObat;
    private double harga;
    private int stok;

    public double getHarga() { return harga; }
    public void setHarga(double harga) {
```

```

        this.harga = harga;
    }
    // field lain tetap private, diakses via getter/setter
}

```

Alasan penggunaan:

Semua field pada class model dideklarasikan private dan hanya dapat diakses atau diubah melalui method getter dan setter publik. Hal ini mencegah kode di luar class mengubah data secara langsung tanpa melalui validasi yang seharusnya.

Manfaat bagi aplikasi:

Data lebih terlindungi dari perubahan yang tidak diinginkan, memudahkan penelusuran bug karena semua perubahan nilai field harus melalui setter, dan memberi ruang untuk menambahkan validasi di dalam setter di kemudian hari tanpa mengubah kode pemanggil.

5. Abstract Class dan Interface

Lokasi implementasi: klinik.model.Person (abstract class) dan klinik.dao.ICrud<T> (interface)

```

public abstract class Person {
    public abstract String getInfo();
}

public interface ICrud<T> {
    void tambah(T data) throws SQLException;
    void ubah(T data) throws SQLException;
    void hapus(int id) throws SQLException;
    List<T> tampilkanSemua() throws SQLException;
    List<T> cari(String kataKunci) throws SQLException;
}

```

Alasan penggunaan:

Person dijadikan abstract class karena secara konsep tidak boleh ada objek “Orang” generik yang berdiri sendiri di klinik ini - yang ada hanyalah Pasien atau Dokter, sehingga Person hanya berfungsi sebagai kerangka bersama. Sementara itu ICrud<T> dijadikan interface generik karena seluruh DAO wajib menyediakan lima operasi standar (tambah, ubah, hapus, tampilkanSemua, cari) tanpa terikat pada satu implementasi tertentu.

Manfaat bagi aplikasi:

Abstract class memaksa subclass melengkapi method getInfo() sesuai konteksnya masing-masing, sedangkan interface ICrud<T> menjaga konsistensi kontrak antar-DAO sehingga seluruh modul CRUD pada aplikasi mengikuti pola yang seragam dan mudah diperluas (misalnya menambah entitas baru cukup membuat DAO baru yang implements ICrud<T>).

6. Package

Lokasi implementasi: struktur folder src/klinik/{model, dao, util, exception, view}

```

klinik.model    -> Person, Pasien, Dokter, User, Obat, Kunjungan, ResepObat
klinik.dao      -> ICrud, PasienDAO, DokterDAO, ObatDAO, KunjunganDAO, UserDAO
klinik.util     -> KoneksiDatabase, Validasi, LogUtil
klinik.exception-> StokObatException, DataTidakDitemukanException

```

```

klinik.view    -> LoginForm, MenuUtama, FormPasien, FormDokter,
                  FormObat, FormKunjungan, LaporanKunjungan

```

Alasan penggunaan:

Kode dipisahkan ke dalam lima package berdasarkan tanggung jawabnya: model (representasi data), dao (akses database), util (fungsi bantu lintas modul), exception (exception khusus aplikasi), dan view (antarmuka Swing). Pemisahan ini mengikuti prinsip separation of concerns.

Manfaat bagi aplikasi:

Struktur project menjadi rapi dan mudah dinavigasi, menghindari konflik penamaan class, serta memudahkan kolaborasi karena setiap package memiliki tanggung jawab yang jelas dan dapat dikembangkan relatif independen.

7. Collection

Lokasi implementasi: klinik.model.Kunjungan (List<ResepObat>) dan seluruh class pada package klinik.dao (ArrayList sebagai hasil query)

```

// Kunjungan.java
private List<ResepObat> daftarResep = new ArrayList<>();

public void tambahResep(ResepObat resep) {
    this.daftarResep.add(resep);
}

public double getTotalBiaya() {
    double totalObat = 0;
    for (ResepObat r : daftarResep) {
        totalObat += r.getSubtotal();
    }
    return biayaPeriksa + totalObat;
}

```

Alasan penggunaan:

Satu kunjungan dapat memiliki banyak resep obat, sehingga hubungan satu-ke-banyak ini secara alami dipetakan menggunakan List<ResepObat> pada class Kunjungan. Selain itu, setiap method tampilkanSemua() dan cari() pada DAO mengembalikan ArrayList karena jumlah baris hasil query bersifat dinamis dan tidak diketahui di awal.

Manfaat bagi aplikasi:

Collection memberi struktur data yang fleksibel (ukuran dapat berubah saat runtime) dibandingkan array biasa, serta menyediakan method siap pakai (add, get, size, iterasi for-each) yang menyederhanakan pengolahan daftar data pasien, obat, maupun resep pada tampilan tabel Swing (JTable).

8. String Method

Lokasi implementasi: klinik.util.Validasi.java

```

public static boolean isNoTelpValid(String noTelp) {
    if (isKosong(noTelp)) return false;
    String bersih = noTelp.trim().replaceAll("[^0-9]", "");
}

```

```

        return bersih.matches("\\d{8,15}");
    }

    public static String kapitalisasi(String teks) {
        String[] kata = teks.trim().toLowerCase().split("\\s+");
        StringBuilder hasil = new StringBuilder();
        for (String k : kata) {
            hasil.append(Character.toUpperCase(k.charAt(0)))
                .append(k.substring(1)).append(" ");
        }
        return hasil.toString().trim();
    }
}

```

Alasan penggunaan:

Setiap input dari form Swing berupa String yang perlu dibersihkan dan divalidasi sebelum disimpan ke database, misalnya menghilangkan spasi berlebih, memastikan field tidak kosong, memvalidasi format nomor telepon, dan menstandarkan kapitalisasi nama. Class Validasi dibuat khusus untuk mengumpulkan seluruh logika ini agar tidak berulang di setiap form.

Manfaat bagi aplikasi:

Data yang tersimpan di database menjadi lebih konsisten dan rapi (misalnya nama pasien selalu berformat kapital di awal kata), serta mengurangi risiko kesalahan input seperti nomor telepon yang berisi huruf atau field wajib yang dikosongkan.

9. Exception Handling

Lokasi implementasi: klinik.exception.StokObatException & DataTidakDitemukanException, digunakan pada ObatDAO.kurangiStok() dan ditangkap pada FormKunjungan.java

```

// ObatDAO.java
public void kurangiStok(int idObat, int jumlah)
    throws SQLException, StokObatException,
        DataTidakDitemukanException {
    // ... cek stok pada database ...
    if (!rs.next()) {
        throw new DataTidakDitemukanException(
            "Obat dengan ID " + idObat + " tidak ditemukan.");
    }
    if (stokSaatIni < jumlah) {
        throw new StokObatException(
            "Stok tidak mencukupi. Sisa stok: " + stokSaatIni);
    }
    // ... update stok jika lolos validasi ...
}

// FormKunjungan.java - penanganan exception
try {
    obatDAO.kurangiStok(idObat, jumlah);
} catch (StokObatException | DataTidakDitemukanException ex) {
    JOptionPane.showMessageDialog(this, ex.getMessage());
}

```

Alasan penggunaan:

Saat transaksi resep obat disimpan, sistem harus memastikan stok obat mencukupi sebelum stok dikurangi. Java sudah menyediakan exception bawaan seperti SQLException, namun kondisi “stok tidak cukup” atau “data obat tidak ditemukan” adalah aturan bisnis klinik yang khusus, sehingga

dibuat dua custom checked exception agar pesan error lebih jelas dan dapat ditangani secara spesifik berbeda dari error database biasa.

Manfaat bagi aplikasi:

Transaksi menjadi lebih aman karena stok obat tidak akan pernah bernilai negatif; pengguna juga mendapat pesan error yang informatif (mis. "Stok tidak mencukupi. Sisa stok: 5") melalui dialog Swing, alih-alih aplikasi crash atau menyimpan data yang tidak valid.

10. File Handling

Lokasi implementasi: klinik.util.LogUtil.java, dipanggil dari LoginForm, MenuUtama, dan form-form CRUD

```
public static void tulisLog(String aktivitas) {
    try (FileWriter fw = new FileWriter(FILE_LOG, true);
        PrintWriter pw = new PrintWriter(fw)) {
        String waktu = LocalDateTime.now()
            .format(DateTimeFormatter.ofPattern("dd-MM-yyyy HH:mm:ss"));
        pw.println "[" + waktu + "] " + aktivitas);
    } catch (IOException e) {
        System.err.println("Gagal menulis log: " + e.getMessage());
    }
}
```

Alasan penggunaan:

Aplikasi perlu mencatat jejak aktivitas penting (login, tambah/ubah/hapus data, transaksi kunjungan) untuk keperluan audit sederhana. LogUtil menuliskan setiap aktivitas ke dalam berkas teks log_aktivitas.txt menggunakan FileWriter dan PrintWriter dalam mode append, dibungkus try-with-resources agar berkas selalu tertutup otomatis walau terjadi error.

Manfaat bagi aplikasi:

Setiap aktivitas pengguna dapat ditelusuri kembali kapan pun dibutuhkan (misalnya untuk mengetahui siapa yang menghapus data pasien tertentu), tanpa perlu tabel log tambahan di database, sekaligus menjadi contoh penerapan I/O file yang aman menggunakan try-with-resources.

11. Database

Lokasi implementasi: seluruh class pada package klinik.dao, serta berkas database/klinik_db.sql

Seluruh akses data (SELECT, INSERT, UPDATE, DELETE) dipisahkan dari tampilan (view) dan diletakkan pada package dao menggunakan JDBC dengan PreparedStatement untuk mencegah SQL Injection. Basis data MySQL klinik_db terdiri atas enam tabel yang saling berelasi melalui foreign key.

Alasan penggunaan: pola DAO (Data Access Object) dipilih agar logika SQL tidak bercampur dengan kode GUI, sehingga setiap form Swing cukup memanggil method DAO tanpa perlu tahu detail query SQL di baliknya.

Manfaat bagi aplikasi: perubahan struktur tabel atau optimasi query cukup dilakukan di satu titik (class DAO terkait) tanpa mengubah kode tampilan, serta PreparedStatement melindungi aplikasi dari celah keamanan SQL Injection.

```
// PasienDAO.java - contoh PreparedStatement (mencegah SQL Injection)
public List<Pasien> cari(String kataKunci) throws SQLException {
    String sql = "SELECT * FROM pasien WHERE nama LIKE ? OR no_rm LIKE ?";
    try (Connection conn = KoneksiDatabase.getConnection();
        PreparedStatement ps = conn.prepareStatement(sql)) {
        ps.setString(1, "%" + kataKunci + "%");
        ps.setString(2, "%" + kataKunci + "%");
        // ... eksekusi query, mapping ResultSet ke object Pasien ...
    }
}
```

C. Refleksi Pengembangan

1. Mengapa memilih studi kasus Sistem Klinik Sederhana?

Studi kasus klinik dipilih karena mencakup seluruh unsur wajib tugas secara alami dalam satu alur bisnis yang saling berkaitan: terdapat data master (pasien, dokter, obat), transaksi (kunjungan dan resep obat), relasi antar tabel yang jelas (satu kunjungan bisa memiliki banyak resep, satu pasien bisa memiliki banyak kunjungan), serta aturan bisnis nyata yang cocok dijadikan contoh Exception Handling, yaitu validasi stok obat. Selain itu, domain klinik cukup dikenal sehari-hari sehingga memudahkan perancangan form dan laporan tanpa perlu riset domain yang rumit.

2. Bagian program yang paling sulit dibuat dan alasannya

Bagian yang paling menantang adalah modul Transaksi Kunjungan (FormKunjungan), karena satu transaksi melibatkan beberapa tabel sekaligus (kunjungan dan resep_obat) yang harus konsisten satu sama lain. Kesulitan utamanya terletak pada penggabungan tiga hal dalam satu alur: (1) validasi stok obat sebelum resep disimpan, (2) perhitungan total biaya yang harus terus diperbarui setiap resep ditambahkan atau dihapus dari daftar sementara, dan (3) penanganan exception ganda (StokObatException dan DataTidakDitemukanException) tanpa membuat transaksi tersimpan sebagian (data tidak konsisten) apabila salah satu resep gagal diproses.

3. Kesalahan (error) yang paling sering ditemui dan cara mengatasinya

Selama pengembangan, error yang paling sering muncul antara lain:

- `ClassNotFoundException / com.mysql.cj.jdbc.Driver` tidak ditemukan - terjadi karena driver MySQL Connector/J belum ditambahkan ke classpath project. Diatasi dengan memastikan file `mysql-connector-j-x.x.x.jar` sudah dimasukkan ke Libraries/Build Path IDE.
- `SQLException: Access denied for user` - terjadi karena username/password pada `KoneksiDatabase.java` belum disesuaikan dengan konfigurasi MySQL lokal. Diatasi dengan menyesuaikan konstanta USER dan PASSWORD pada class tersebut.
- `NullPointerException` pada `JTable` - terjadi ketika data hasil query kosong namun tetap coba ditampilkan atau diproses. Diatasi dengan menambahkan pengecekan data kosong sebelum memproses hasil `ResultSet`.
- Data tidak konsisten antara stok obat dan resep - awalnya stok dapat berkurang meskipun input jumlah melebihi stok tersedia. Diatasi dengan menambahkan custom exception `StokObatException` yang memvalidasi stok sebelum proses update dijalankan.

4. Tiga fitur yang ingin ditambahkan jika dikembangkan kembali

1. Enkripsi password (misalnya menggunakan hashing BCrypt) menggantikan penyimpanan password polos (plain text) yang saat ini masih digunakan pada tabel users, demi meningkatkan keamanan aplikasi.
2. Fitur cetak struk/laporan ke dalam format PDF (misalnya menggunakan library iText) sehingga laporan kunjungan dan biaya dapat langsung dicetak atau dikirim ke pasien.

3. Manajemen hak akses (role-based access control) yang lebih detail, misalnya membatasi menu tertentu (seperti hapus data atau laporan keuangan) hanya dapat diakses oleh role admin, tidak oleh petugas.

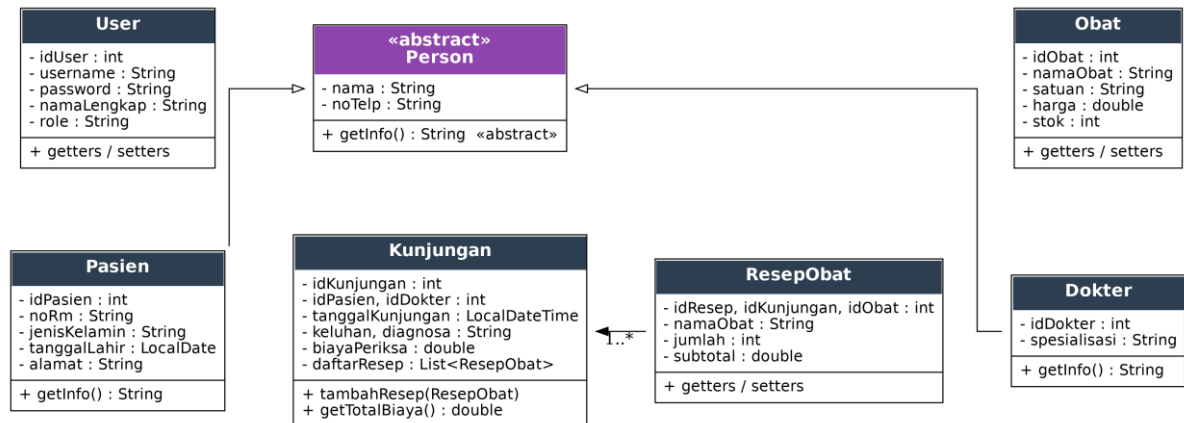
5. Konsep PBO yang masih perlu dipelajari lebih lanjut

Konsep yang dirasa masih perlu didalami lebih lanjut adalah desain pola arsitektur lanjutan seperti penerapan design pattern (misalnya MVC secara lebih ketat, Factory Pattern, atau Singleton untuk koneksi database), serta penggunaan Generics secara lebih luas di luar interface ICrud<T> yang sudah diterapkan, agar kode semakin fleksibel dan mudah diuji (unit testing).

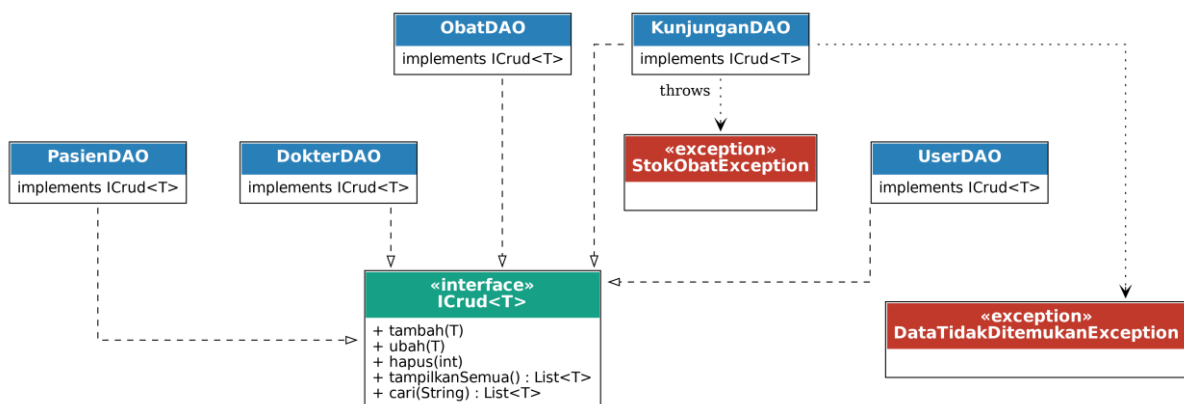
D. Lampiran

1. Class Diagram

Diagram berikut dipisah menjadi dua bagian agar setiap class tetap terbaca jelas: (a) lapisan model yang menggambarkan hubungan pewarisan (inheritance) antara Person dengan Pasien dan Dokter, serta komposisi antara Kunjungan dan ResepObat; dan (b) lapisan DAO yang menggambarkan hubungan realisasi (implements) antara interface ICrud<T> dengan kelima DAO, serta relasi DAO dengan custom exception.

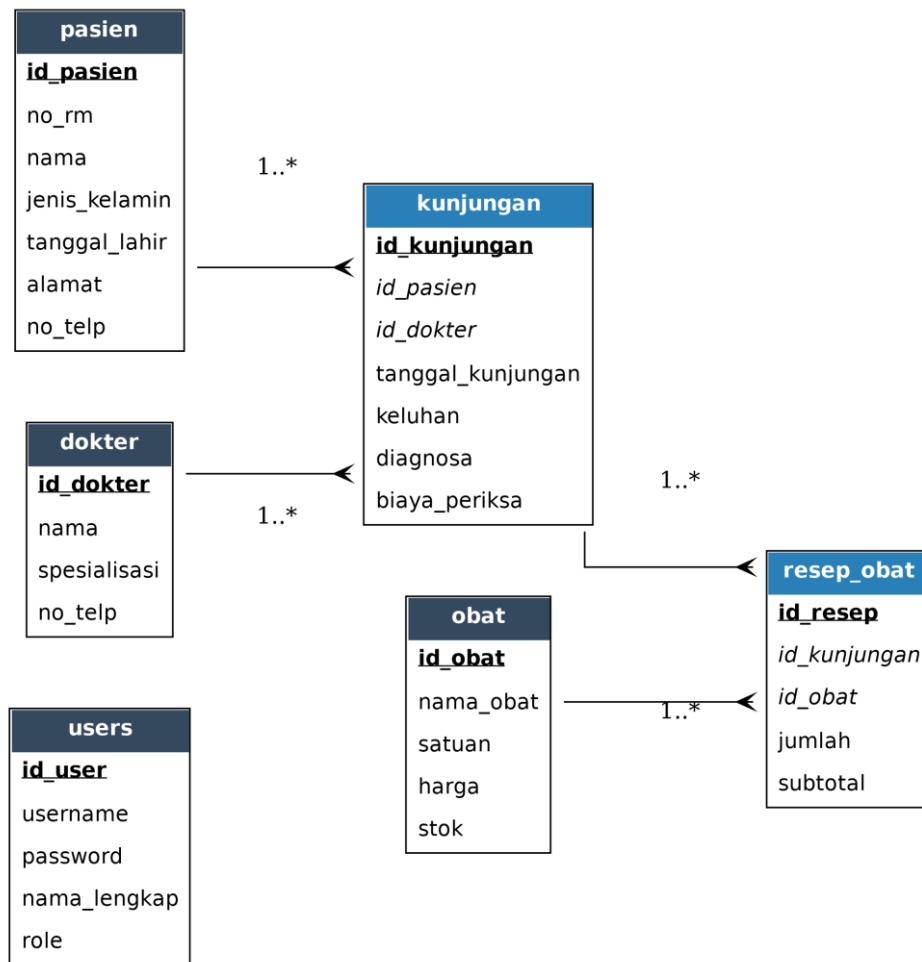


Gambar 1a. Class Diagram - Lapisan Model (Person, Pasien, Dokter, User, Obat, Kunjungan, ResepObat)



Gambar 1b. Class Diagram - Lapisan DAO, Interface ICrud<T>, dan Custom Exception

2. Struktur Database



Gambar 2. Entity Relationship Diagram (ERD) klinik_db

Tabel users

Kolom	Tipe Data	Keterangan
id_user	INT (PK, AUTO_INCREMENT)	Primary key pengguna
username	VARCHAR(50) UNIQUE	Nama login, harus unik
password	VARCHAR(100)	Kata sandi login
nama_lengkap	VARCHAR(100)	Nama lengkap pengguna
role	ENUM('admin','petugas')	Level akses pengguna

Tabel pasien

Kolom	Tipe Data	Keterangan
id_pasien	INT (PK, AUTO_INCREMENT)	Primary key pasien
no_rm	VARCHAR(20) UNIQUE	Nomor rekam medis
nama	VARCHAR(100)	Nama pasien
jenis_kelamin	ENUM('L','P')	Jenis kelamin
tanggal_lahir	DATE	Tanggal lahir pasien
alamat	VARCHAR(255)	Alamat pasien
no_telp	VARCHAR(20)	Nomor telepon pasien

Tabel dokter

Kolom	Tipe Data	Keterangan
id_dokter	INT (PK, AUTO_INCREMENT)	Primary key dokter
nama	VARCHAR(100)	Nama dokter
spesialisasi	VARCHAR(100)	Bidang spesialisasi
no_telp	VARCHAR(20)	Nomor telepon dokter

Tabel obat

Kolom	Tipe Data	Keterangan
id_obat	INT (PK, AUTO_INCREMENT)	Primary key obat
nama_obat	VARCHAR(100)	Nama obat
satuan	VARCHAR(20)	Satuan obat (strip/botol)
harga	DECIMAL(10,2)	Harga per satuan
stok	INT	Jumlah stok tersedia

Tabel kunjungan

Kolom	Tipe Data	Keterangan
id_kunjungan	INT (PK, AUTO_INCREMENT)	Primary key kunjungan
id_pasien	INT (FK -> pasien)	Relasi ke tabel pasien
id_dokter	INT (FK -> dokter)	Relasi ke tabel dokter
tanggal_kunjungan	DATETIME	Waktu kunjungan
keluhan	TEXT	Keluhan pasien
diagnosa	TEXT	Diagnosa dokter
biaya_periksa	DECIMAL(10,2)	Biaya pemeriksaan dasar

Tabel resep_obat

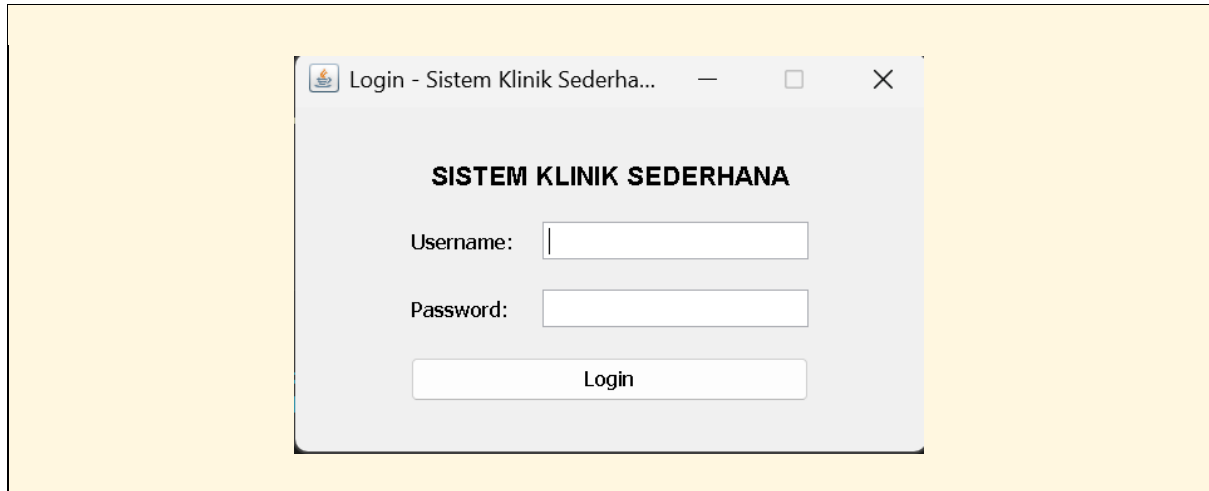
Kolom	Tipe Data	Keterangan
id_resep	INT (PK, AUTO_INCREMENT)	Primary key resep
id_kunjungan	INT (FK -> kunjungan)	Relasi ke tabel kunjungan
id_obat	INT (FK -> obat)	Relasi ke tabel obat
jumlah	INT	Jumlah obat diresepkan
subtotal	DECIMAL(10,2)	Subtotal harga obat

3. Screenshot Aplikasi dan Penjelasan Menu

Catatan: sertakan tangkapan layar (screenshot) aplikasi yang benar-benar dijalankan di komputer Anda pada kotak bertanda di bawah ini. Jalankan aplikasi (klinik.Main) setelah database MySQL aktif, lalu tangkap layar setiap menu berikut.

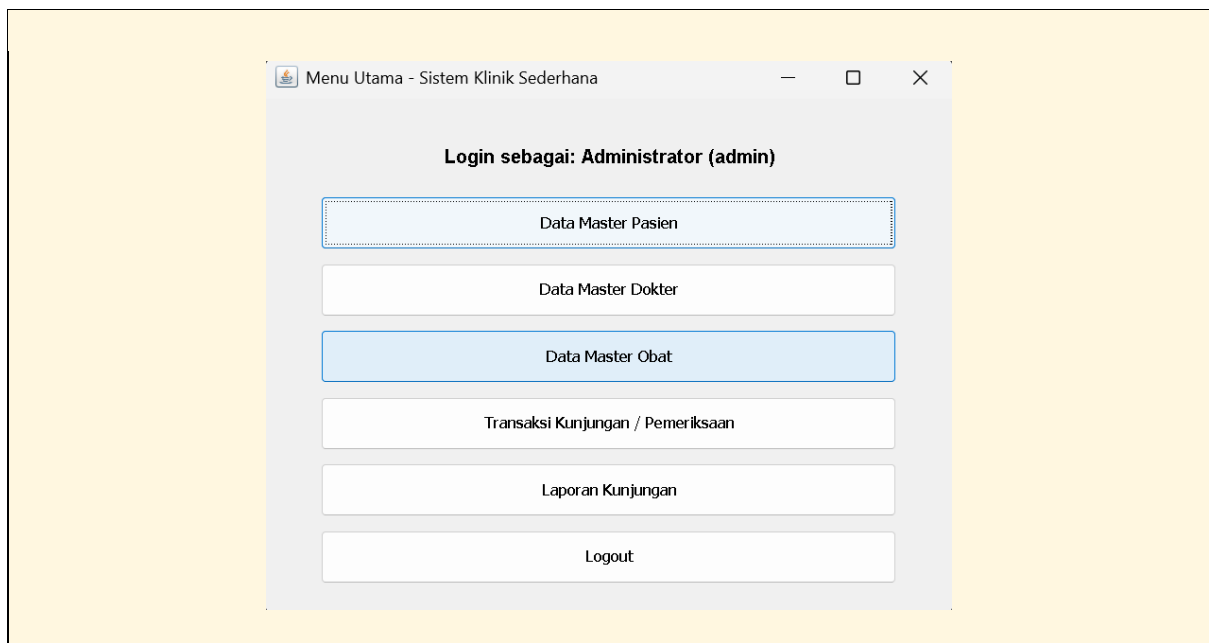
a. Form Login

Form login menampilkan input username dan password. Sistem memeriksa kecocokan data melalui UserDao ke tabel users, kemudian mengarahkan pengguna ke Menu Utama apabila kredensial valid, disertai pencatatan aktivitas login ke berkas log.



b. Menu Utama

Menu Utama menampilkan informasi pengguna yang sedang login beserta enam tombol navigasi: Data Master Pasien, Data Master Dokter, Data Master Obat, Transaksi Kunjungan/Pemeriksaan, Laporan Kunjungan, dan Logout.



c. Form Data Master Pasien

Form ini menyediakan operasi CRUD lengkap (tambah, ubah, hapus, tampilkan) untuk data pasien beserta fitur pencarian berdasarkan nama atau nomor rekam medis, memanfaatkan PasienDAO dan class Validasi untuk memastikan data yang diinput valid.

ID	No RM	Nama	JK	Tgl Lahir	Alamat	Telp
2	RM0002	Rina Wula...	P	1998-08-21	Jl. Sudirma...	081298765...
1	RM0001	Andi Saputra	L	1995-05-10	Jl. Merdeka...	081234567...

d. Form Data Master Dokter

Form ini digunakan untuk mengelola data dokter (nama, spesialisasi, nomor telepon) dengan operasi CRUD dan pencarian yang serupa dengan form pasien, melalui DokterDAO.

ID	Nama	Spesialisasi	Telp
2	dr. Maya Kartika	Anak	08123334444
1	dr. Budi Santoso	Umum	081211112222

e. Form Data Master Obat

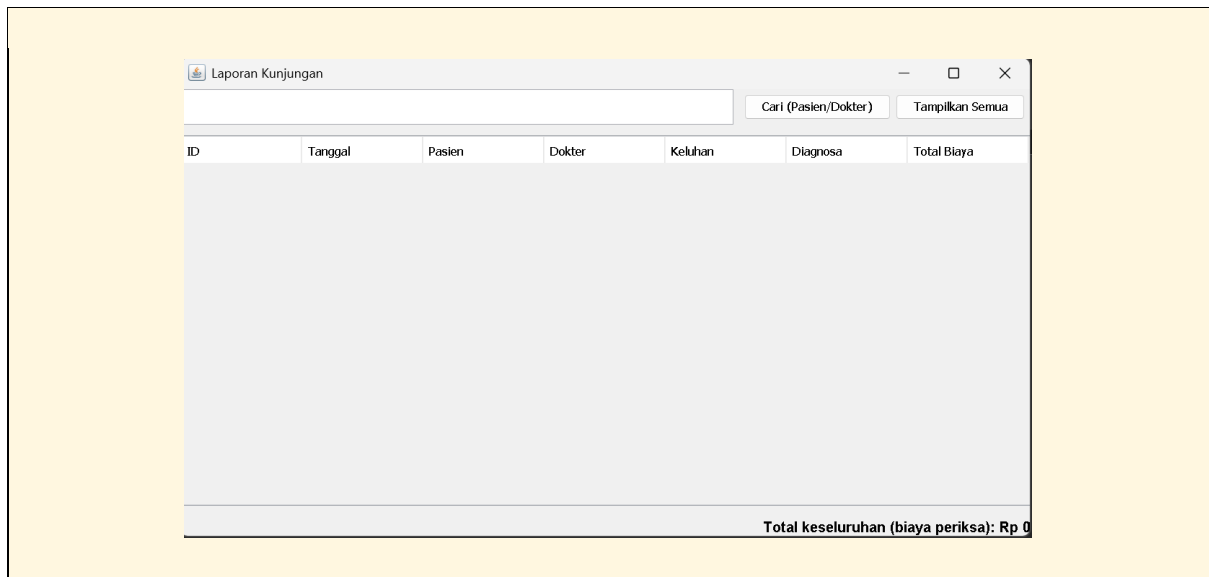
Form ini mengelola data obat beserta stok dan harga. Perubahan stok pada form ini akan memengaruhi validasi stok saat transaksi resep obat dilakukan pada form kunjungan.

f. Form Transaksi Kunjungan

Form ini merupakan inti transaksi aplikasi: memilih pasien dan dokter, mencatat keluhan dan diagnosa, menambahkan satu atau lebih resep obat (yang divalidasi ketersediaan stoknya), serta menghitung total biaya kunjungan secara otomatis (biaya periksa + total harga obat).

g. Laporan Kunjungan

Menu ini menampilkan rekap data kunjungan pasien dalam bentuk tabel, memungkinkan petugas melihat riwayat kunjungan, diagnosa, dan total biaya setiap transaksi.



Penutup

Demikian Laporan Analisis Program ini disusun sebagai bagian dari pemenuhan Tugas Remedial Mata Kuliah Pemrograman Berbasis Objek (Java). Laporan ini menjelaskan bagaimana aplikasi Sistem Klinik Sederhana dibangun dengan menerapkan konsep-konsep PBO secara menyeluruh, mulai dari Class dan Object hingga integrasi dengan basis data MySQL, disertai refleksi proses pengembangan yang telah dilalui.